

■ Debugging in React Apps

A beginner-friendly guide to finding and fixing bugs in React

Bugs are a normal part of writing code. React gives you several powerful tools to find and fix problems quickly. This guide walks you through the most important debugging techniques — step by step, in plain English.

■ 1. Understanding React Error Messages

What are Error Messages?

When something goes wrong in your React app, React shows an error message in the browser. These messages can look scary at first, but they are actually very helpful — they tell you **what went wrong** and **where it happened**.

Parts of a React Error Message

- **Error type** — e.g. `TypeError`, `ReferenceError`, or a custom React error.
- **Error description** — a short sentence describing the problem.
- **Component stack** — the list of components where the error occurred.
- **File & line number** — the exact file and line to look at.

Common Error Examples

```
TypeError: Cannot read properties of undefined (reading "map")  
  → You are calling .map() on a variable that is undefined.  
  Fix: Check that your data is loaded before using it.  
  
Warning: Each child in a list should have a unique "key" prop.  
  → Add a key={item.id} to each item in your list.
```

■ Tip

Always read the error message from top to bottom. The first line tells you what went wrong. The stack trace below it tells you where.

■ 2. Using the Browser Debugger & Breakpoints

What is the Browser Debugger?

Every modern browser (Chrome, Firefox, Edge) has built-in developer tools that include a **debugger**. The debugger lets you **pause your code** at any line and inspect what is happening — variable values, function calls, and more.

How to Open Developer Tools

- Press **F12** or **Ctrl + Shift + I** (Windows/Linux)
- Press **Cmd + Option + I** (Mac)

- Right-click anywhere on the page → click **Inspect**

What are Breakpoints?

A **breakpoint** is a marker you place on a line of code. When your app reaches that line, it pauses — so you can look at all the values at that exact moment.

How to Set a Breakpoint

- Open DevTools → click the **Sources** tab.
- Find your file in the file tree on the left.
- Click on a **line number** — a blue marker appears. That is your breakpoint.
- Reload the page. The code will pause at that line.

You can also use the debugger keyword in your code:

```
function handleClick() {  
  debugger; // Code pauses here when this runs  
  console.log('Button clicked');  
}
```

■ Tip

While paused, hover over any variable to see its current value. Use the buttons Step Over, Step Into, and Resume to move through the code one line at a time.

■■ 3. Understanding React's Strict Mode

What is Strict Mode?

React's **Strict Mode** is a special wrapper you can put around your app (or part of it). It does **not change how your app looks** to users. Instead, it activates extra checks and warnings in development to help you write better code.

How to Enable Strict Mode

Wrap your app (or any component) with `<React.StrictMode>`:

```
import React from 'react';  
import ReactDOM from 'react-dom/client';  
import App from './App';  
  
ReactDOM.createRoot(document.getElementById('root')).render(  
  
  
);
```

What Does Strict Mode Do?

- **Double-invokes functions** — calls render and some lifecycle methods twice to catch side effects.
- **Warns about deprecated APIs** — alerts you if you use old React features that will be removed.
- **Detects unexpected side effects** — helps you find code that should not run during rendering.

■ Tip

Strict Mode only runs in development mode. It has zero impact on your production build — so always leave it on!

■ 4. Using the React DevTools (Browser Extension)

What is React DevTools?

React DevTools is a free browser extension made by the React team. It adds two new tabs to your browser's developer tools — **Components** and **Profiler** — giving you a live view inside your React app.

How to Install React DevTools

- Open your browser's extension store (Chrome Web Store / Firefox Add-ons).
- Search for **React Developer Tools**.
- Click **Add to Browser** and confirm.
- Open any React app and press F12 — you will see the new tabs.

The Components Tab

This tab shows you the **component tree** of your app — like a family tree of all your components. Click on any component to see:

- **Props** — the data passed into that component.
- **State** — the internal data the component is managing.
- **Hooks** — the hooks being used, like `useState` and `useEffect`.

The Profiler Tab

The Profiler tab helps you find **performance problems**. Record a session and it shows which components re-rendered, how often, and how long they took.

■ Tip

You can edit props and state directly inside the Components tab to test how your component behaves — no code changes needed! Great for quick experiments.

Quick Summary

Tool / Feature	What It Helps With
Error Messages	Know what broke and where
Browser Debugger & Breakpoints	Pause code and inspect values
React Strict Mode	Catch bad patterns early
React DevTools Extension	Inspect components, props & state

Happy Debugging! ■ — With the right tools, every bug is solvable.